

CC451: Interaccion Humano Computadora

Laboratorio 05: Prototipado e Inferencia Multimodal de Experiencias de Usuario con Google AI Studio y Gemini API

1. INFORMACIÓN GENERAL

- **Duración:** 3 horas (180 minutos).
- **Enfoque Pedagógico:** Aprendizaje Basado en Problemas (ABP), Centrado en el Usuario y Usabilidad de Sistemas Basados en IA (IA UX / Generative UI).
- **Entregable:** Archivo comprimido CC451_Lab05_[Código]_[Nombre_Apellido].zip conteniendo prompts exportados, scripts de integración en Python y reporte cualitativo.

2. OBJETIVOS DE APRENDIZAJE

Al finalizar la sesión, el estudiante de ingeniería será capaz de:

1. **Evaluar el Impacto de Parámetros de Modelo en UX:** Comprender de forma experimental cómo la variabilidad (temperatura) y el control de salida (JSON Estructurado, Grounding) afectan la usabilidad y consistencia de las respuestas.
2. **Diseñar e Iterar Prompts Multimodales (System Instructions & Few-Shot):** Prototipar interactores asistivos analizando imágenes, contexto visual y necesidades explícitas de usuarios.
3. **Desarrollar y Exportar Prototipos Funcionales a Código:** Integrar la llamada a la API de Gemini mediante Python para generar respuestas de audio/texto enfocadas en accesibilidad e interacción multimodal.

3. PRERREQUISITOS Y RECURSOS

- **Google AI Studio IDE:** aistudio.google.com
- **Entorno de desarrollo Python:** Google Colab o Jupyter Notebook con paquete google-genai actualizado.
- **Documentación Oficial:**
 - [AI Studio Quickstart](#)
 - [Guía de Prompts Multimodales](#)

4. ESTRUCTURA Y CRONOGRAMA DE DESARROLLO (180 MINUTOS)

BLOQUE 1: EXPLORACIÓN TÉCNICA Y HCI EN GOOGLE AI STUDIO (30 MIN)

1.1 Configuración e Interfaz

1. Accede a **Google AI Studio** y selecciona el modelo **gemma-4-26b-a4b-it**
2. Identifica los tres tipos de mensajes principales:
 - **Freeform / Chat Prompts:** Interacciones libres en vivo .
 - **System Instructions:** Definen el rol, restricciones y tono del sistema.
 - **Structured Outputs / JSON Mode:** Aseguran respuestas parseables por software.

1.2 Laboratorio de Temperatura y Carga Cognitiva

- Realiza una prueba cargando una imagen compleja (ej. la Catedral de Lima o el Convento de los Descalzos)[cite: 2, 4].
- Configura la **System Instruction:** *"Actúa como un guía turístico experto. Identifica el lugar y explica brevemente su valor arquitectónico e histórico."*
- **Experimento de Usabilidad:**
 1. Ejecuta el prompt con **Temperatura = 0.0**. Observa la precisión técnica e inmutabilidad.
 2. Ejecuta el prompt con **Temperatura = 1.0**. Analiza las variaciones del lenguaje natural.
 3. **Reflexión HCI:** Explica en 2 líneas en qué escenarios de interfaz prefieres una temperatura baja (ej. diagnóstico de errores, accesibilidad) frente a alta (ej. generación creativa, lluvia de ideas).

BLOQUE 2: PROTOTIPADO CASO DE USO 1 — MESERO ROBÓTICO AUTÓNOMO (75 MIN)

Contexto de UX:

Un robot interactivo atiende en el salón de un restaurante. No solo responde a texto; debe inferir el estado físico del salón y las emociones no verbales de los comensales a partir de sensores visuales para determinar si se requiere su intervención[cite: 4, 5].

Instrucciones:

Debes diseñar e iterar **2 Prompts Multimodales Avanzados** en AI Studio :

- **Prompt 1 (Análisis Espacial y Requerimiento de Intervención):**
- **Input:** Imagen de una mesa con platos, vasos o interacción social.
- **Objetivo:** Identificar los objetos clave, clasificar el estado de la mesa (limpia, desordenada, platos terminados) y tomar una **decisión binaria sobre intervención** (Intervenir: Sí/No) con su correspondiente justificación.
- **Optimización de Prompt:** Implementa una técnica de *Chain-of-Thought* (Pensamiento Paso a Paso) en el prompt para evitar falsas intervenciones que incomoden al cliente.
- **Prompt 2 (Diagnóstico Empático y Oferta de Servicio):**
- **Input:** Fotografías de comensales en diferentes situaciones emocionales (buscando al mozo, aburridos, festejando, indecisos).
- **Objetivo:** Analizar la postura/expresión y generar una sugerencia de servicio adecuada, respetuosa y oportuna en formato estructurado (JSON).

Evaluación y Benchmarking:

Prueba cada uno de tus 2 prompts optimizados con al menos **3 imágenes distintas** .
Genera una matriz de evaluación de calidad de respuesta según la escala de usabilidad:

Ejemplo	Prompt Base	Prompt Optimizado	Calificación (1-5)	Justificación de la Mejora (HCI/UX)
Imagen 1 (Mesa con vasos vacíos)	"Dime qué hay en la mesa"	"Analiza los objetos en la mesa. Clasifica... [Prompt Optimizado]"	5/5	Reduce la ambigüedad y evita respuestas irrelevantes .
Imagen 2 (Comensal levantando la mano)	...	...	...	...
Imagen 3 (Familia celebrando)	...	...	...	...

BLOQUE 3: PROTOTIPADO CASO DE USO 2 — SISTEMA ASSISTIVE VISION (60 MIN)

Contexto de Accesibilidad :

Un sistema de asistencia para personas con discapacidad visual captura imágenes continuas desde una cámara montada en unas gafas inteligentes. La información debe ser concisa, priorizar la seguridad física (evitar obstáculos) y transmitirse mediante voz sintetizada (TTS).

Actividades:

1. **Mapa de Entornos Críticos (Análisis HCI):** Identifica y argumenta 3 entornos cotidianos donde un usuario invidente enfrenta riesgos o barreras arquitectónicas (ej. cruce peatonal sin semáforo sonoro, andén de transporte público, pasillo de supermercado).

2. Diseño de Prompt de Asistencia Inmediata (Prompt 3):

- Diseña un prompt multimodal de alta urgencia/prioridad.
- **Reglas de UI por Voz:**
- Priorizar obstáculos inminentes primero.
- Usar referencias relativas espaciales claras ("a tu izquierda a 1 metro", "al frente a la altura de la cabeza").
- Mantener respuestas por debajo de 25 palabras para no saturar el canal auditivo del usuario.

3. Integración mediante Python Script (API + Audio/TTS):

- Exporta el código desde la opción **Get Code** de Google AI Studio.
- Completa la implementación en un archivo ejecutable en Python (asistente_invidente.py):

Python

```
import os
from google import genai
from google.genai import types

# Inicializar cliente de Gemini API
client = genai.Client(api_key=os.environ.get("GEMINI_API_KEY"))

def analizar_entorno_asistivo(ruta_imagen):
    # Cargar archivo de imagen
    image_file = client.files.upload(file=ruta_imagen)
```

```
system_instruction = ""
```

Eres un asistente de navegación por voz para una persona con discapacidad visual.

Reglas estrictas de interacción:

1. Menciona primero riesgos inmediatos de seguridad.
2. Usa orientación espacial basada en reloj o direcciones (ej. 'a tus 12').
3. Mantiene la respuesta por debajo de 25 palabras. Sé directo y claro.

```
"""
```

```
response = client.models.generate_content(  
    model="gemini-2.5-flash",  
    contents=[image_file, "Analiza este entorno y dame la indicación de navegación urgente."],  
    config=types.GenerateContentConfig(  
        system_instruction=system_instruction,  
        temperature=0.2  
    )  
)  
return response.text
```

```
# Ejecución de prueba
```

```
# print(analizar_entorno_asistivo("obstaculo.jpg"))
```

BLOQUE 4: ENTREGABLE Y EVALUACIÓN (15 MIN)

Estructura del Archivo de Entrega (CC451_Lab05_[Nombre_Apellido].zip):

Plaintext

CC451_Lab05_20241234_Juan_Perez/

├─ Prompts/

| └─ Prompt1_Mesero_Estructurado.json

| └─ Prompt2_Emociones_JSON.json

| └─ Prompt3_Asistente_Visión.json

├─ Capturas/

| └─ AIStudio_Pruebas_Mesero_1..3.png

| └─ AIStudio_Pruebas_Invidente_1..3.png

├─ Scripts/

| └─ laboratorio_prompts.ipynb (o script .py)

| └─ asistente_invidente.py

└─ Reporte_HCI.pdf (Tabla de evaluación de prompts, análisis de entornos y justificación de usabilidad)

5. RÚBRICA DE EVALUACIÓN (TOTAL: 20 PUNTOS)

Criterio	Excelente (100%)	Satisfactorio (75%)	En Proceso (50%)	Deficiente (0%)
Diseño y Optimización de Prompts (HCI) (6 pts)	Aplica técnicas avanzadas (<i>System Instructions, Chain-of-Thought, JSON Output</i>). Argumenta adecuadamente la reducción de carga cognitiva .	Diseña prompts funcionales pero con iteración e inferencia de usabilidad limitada.	Prompts básicos sin optimización clara ni uso de parámetros avanzados.	No realiza las iteraciones de prompts requeridas .
Matriz de Pruebas y Benchmarking (4 pts)	Evalúa con 3+ escenarios por caso . Justifica las puntuaciones en función de la experiencia de usuario y la precisión .	Evalúa 3 casos pero la justificación de usabilidad es superficial.	Incompleto (menos de 3 pruebas o sin tabla comparativa) .	No incluye matriz de pruebas .
Integración de Código y API Python (6 pts)	El script ejecuta correctamente la API de Gemini, procesa imágenes, aplica las reglas del prompt y maneja salidas para TTS/audio.	Código funcional pero con errores menores o sin estructura modular.	Código incompleto que no logra conectarse con la API de Gemini.	No presenta código ejecutable.
Análisis de Accesibilidad y Requisitos (4 pts)	Identifica entornos de alto valor para usuarios invidentes, aplicando principios universales de UX (concisión, orientación espacial).	Identifica entornos válidos pero las respuestas de la IA son demasiado extensas.	Identificación deficiente de entornos o ignorando directrices de UX.	No aborda la sección de accesibilidad.